Kvalificēts elektroniskais paraksts
RENĀRS SUSEJS
2026-04-14 18:44:43 GMT+3
Mērķis: Paraksts
Powered by Dokobit

RĪGAS TEHNISKĀ UNIVERSITĀTE

Rēzeknes akadēmija

Inženierzinātņu centrs

Renārs Susejs

Profesionālā bakalaura studiju programmas “Datorsistēmas” students, stud. apl. nr. rs22060

PIRMSDIPLOMA PRAKSES ATSKAITE

Prakses vadītājs Mg.sc.comp., pasniedzējs

Ilmārs Apeināns

RĒZEKNE 2026

SATURS

IEVADS 3

1. VISPĀRĪGĀ INFORMĀCIJA PAR PRAKSES VIETU 4

2. REALIZĒTO PROGRAMMATŪRAS RISINĀJUMU RAKSTUROJUMS 5

2.1. Finanšu datu saskaņošanas uzlabojumi 7

2.1.1. Izmaiņas datu bāzē 7

2.1.2. Cauruļvada izstrāde Azure Data Factory platformā 8

2.1.3. Procesa iespējošana procesu orķestratorā 10

2.2. Atkārtotā mēģinājuma automatizācija 10

2.2.1. Powershell runbook pielāgošana 12

2.2.2. Mākoņa infrastruktūras izstrāde 12

2.2.3. Jaunas programmatūras palaišana mākoņa infrastruktūrā 13

SECINĀJUMI UN PRIEKŠLIKUMI 15

IZMANTOTĀS LITERATŪRAS UN AVOTU SARAKSTS 17

PIELIKUMI 18

IEVADS

Laikā no 2026. gada 26. janvāra līdz 2024. gada 22. martam, es izgāju praksi uzņēmumā If P&C Insurance AS Latvijas filiāle. Prakses vadītājs RTU Rēzeknes akadēmijā ir Imants Zarembo. Prakses vadītājs uzņēmumā ir Alvis Reinbergs, kas veic programmatūras izstrādātāja un Rezervju domēna vadītāja pienākumus If P&C Insurance AS Latvijas filiālē.

Prakses mērķis:

- Veikt uzlabojumus prakses vietas esošajā sistēmā, ieviešot jaunu funkcionalitāti naudas plūsmu saskaņošanā un veidojot risinājumu sistēmas atsevišķu komponentu noturības pret traucējumiem (resilience) uzlabošanai.

Lai sasniegtu iepriekš izvirzītos mērķus, tika noteikt uzdevumi, kuri palīdzētu tos sasniegt:

- Dokumentēt savu pieredzi un darba laikā veikt pierakstus;
- Pastāvīgi komunicēt ar savu prakses vadītāju uzņēmuma un lūgt atsauksmes par veikto darbu izpildes kvalitāti;
- Iepazīties ar prakses vietas sistēmas dokumentāciju.

Prakses atskaite sastāv no ievada, uzņēmuma raksturojuma, veikto darbu apraksta ar divām nodaļām un astoņām apakšnodaļām, kurās ir aprakstīti izstrādātie un piegādātie risinājumi, kā arī secinājumi un priekšlikumi, deviņi pielikumi un izmantotās literatūras saraksts, kurā ir pieminēti avoti, kuri tika izmantoti prakses laikā, lai veiktu uzdotos darbus un papildinātu trūkstošās zināšanas.

1. VISPĀRĪGĀ INFORMĀCIJA PAR PRAKSES VIETU

If P&C Insurance AS Latvijas filiāle ir apdrošināšanas kompānija, kas piedāvā pilnu riska apdrošināšanas klāstu gan privātpersonām, han uzņēmumiem Zviedrijā, Norvēģijā, Somijā, Dānijā un Baltijas valstīs. Tā ir daļa no If P&C Insurance Holding Ltd., kas savukārt ir daļā no apdrošināšanas grupas Sampo Group. If P&C Insurance tika dibināts 1999. gadā. Kopš 2019. gada If izpilddirektors ir Morten Thorsrud un galvenais birojs atrodas Stokholmā Zviedrijā.

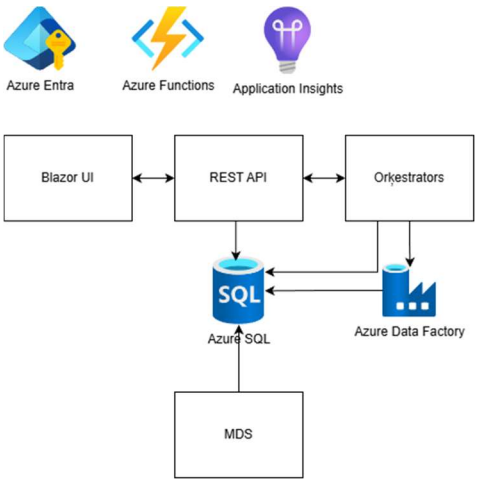
If, kā Sampo grupas uzņēmums, uzsāka savu darbību 1998. gadā un uz doto brīdi darbinieku skaits Latvijā pārsniedz 300. If birojs atrodas Rīgā, Republikas laukumā 2A. If P&C Insurance AS Latvijas filiāles direktors ir Andris Morozovs.

Uzņēmums izmanto dažādus pielāgotus IT risinājumus savā darbā, piemēram intranetu, kas satur visu nepieciešamo informāciju darbiniekam, cilvēkresursu vadības rīkus – Workday, kā arī Microsoft Office programmas. Kaut arī primārā darbības joma ir apdrošināšana, tomēr uzņēmums sevi pozicionē arī kā tehnoloģiju uzņēmumu Latvijā, no 600 darbiniekiem aptuveni 400 ir IT speciālisti un visā uzņēmumā (ieskaitot Ziemeļvalstīs esošos darbiniekus) IT speciālistu skaits sasniedz 2000 cilvēkus no 10000 kopējā darbinieku skaita.

2. REALIZĒTO PROGRAMMATŪRAS RISINĀJUMU RAKSTUROJUMS

Prakses laikā es piedalījos projektā, kura mērķis ir izstrādāt un uzlabot sistēmu, ko izmanto uzņēmuma aktuāri, lai veiktu aprēķinus un noteiktu nepieciešamās finanšu rezerves uzņēmumam. Šis process tiek cieši regulēts ar likumu Solvency II, kas nosaka uzņēmuma maksātspējas nosacījumus, un IFRS grāmatvedības standartu. Sistēma sastāv no vairākām aplikācijām, kas nodrošina to, lai milzīgais finanšu datu daudzums tiktu apstrādāts laikā. If P&C Insurance būdams Microsoft partneris, izmanto Microsoft izstrādātās tehnoloģijas.

Aplikācijas tiek rakstītas, izmantojot C# programmēšanas valodu un .NET ietvaru gan lietotāja grafiskajām saskarnēm, ko nodrošina Blazor ietvars, gan arī servera izstrādei, kas balstās uz ASP.NET un .NET Core. Papildus kodētajām aplikācijām, plaši tiek izmantotas Azure mākoņpakalpojumu sniegtās iespējas, piemēram datu bāzei tiek izmantots Azure SQL serveris, Azure Functions, Azure Automation Account, Azure Data Factory un Azure Service Bus. Tā kā datu bāze ir Azure SQL, tad plaši tiek izmantota T-SQL, kuras pamatā ir strukturētā vaicājumu valoda jeb SQL, kas ļauj rakstīt kodu procedurāli. Sistēmas vajadzībā tiek izmantotas 5 datu bāzes – konfigurāciju, žurnalēšanas, datu noliktavas, starpapstrādes un galvenā operatīvā datu bāze.



1. attēls. Projekta sistēmas shēma.

Projekta pārvaldībai tiek izmantots Azure DevOps serviss, kas ļauj dokumentēt sistēmu, glabāt kodu un visas tā versijas, veidot funkciju un lietotājstāstu biļetenus un sekot līdzi projektam, glabā testēšanas informāciju tai skaitā testa plānus un instrukcijas, kā arī automātisko kompilēšanu un palaišanu dažādās vidēs. Projektā ir 5 vides – izstrādes, sistēmtestēšanas, pieņemšanas testu, integrācijas testēšanas un produkcijas vide.

Šai sistēmai ir atkarības no citām sistēmām, piemēram finanšu datu noliktava, kas apkopo visus apdrošināšanas finanšu transakcijas datus – apdrošināšanas prēmijas iemaksas, pieteikumu atlīdzību izmaksas, kā arī pārapdrošināto pieteikumu dati.

Komanda sastāv no 9 cilvēkiem, kas izstrādā, uztur sistēmu no 4 aplikācijām un mākoņskaitļošanas infrastruktūras, un atbalsta lietotājus. Komanda sastāv 4 izstrādātājiem, 2 kvalitātes inženieriem, 1 biznesa analītiķa, 1 projektu vadītāja un 1 produkta īpašnieks.

2.1. Finanšu datu saskaņošanas uzlabojumi

Katru periodu, kad tiek veikta uzskaites perioda slēgšana, sistēmā tiek laists process, kas salīdzina finanšu datus ar to, kas ir virsgrāmatā (elektroniska grāmatvedības uzskaite) un gadījumā, ja ir atšķirības tad veic pārrēķinu un labojumus. Tā kā visi aprēķini un grāmatojumi rezervēm ir jāveic pēc iespējas ātrāk, tad kļūdu un rezervju uzlabojumu savlaicīgai pamanīšanai bija nepieciešams izstrādāt cauruļvadu (pipeline) Azure Data Factory platformā, kas veiktu saskaņošanu pēc saskaņojumu nosūtīšanas uz virsgrāmatu, lai aktuāri varētu veikt labojumus nākamajās perioda slēgšanas dienās. Cauruļvada darbības rezultātā bija jāuzrādās saskaņošanas rezultātu tabulā ar speciāliem marķieriem, kas norādīja, ka šie ir analītiskie ieraksti, nevis tie, kas tiek nosūtīti uz virsgrāmatu.

Tehnisko risinājumu izveidoja vecākais programmētājs un mans uzdevums bija to ieviest. Ieviešana sastāvēja no jaunas kolonnas pievienošanas un aizpildīšanas, cauruļvada izstrādes Azure Data Factory platformā, kā arī soļa iespējošanas esošajā orķestrēšanas sistēmā.

2.1.1. Izmaiņas datu bāzē

Izmaiņu veikšana datu bāzē krasi atšķiras no tās, ko veic, mainot aplikācijas kodu. Ja aplikācijā izmaiņu ieviešanai pietiek ar izmaiņu veikšanu, veiksmīgu kompilāciju un izmitināšanu, kas bieži tiek paveiktas ar automātisko rīku izmantošanu, tad katra izmaiņa datu bāzē ir jāveic ļoti uzmanīgi tā, lai:

- Datu bāzē nepazustu vai netiktu izmainīti dati;
- Izmaiņu veikšanai ir jābūt iespējamai uz esošajām tabulām, nedzēšot tabulas;
- Izmaiņu rezultātā netiktu traucēts ierastais biznesa process (piemēram, lai nerastos ātrdarbības traucējumi).

Datu bāzes izstrādei ir izveidots speciāls projekts Visual Studio IDE, kurā ir aprakstīti visi datu bāzes objekti, kas ir pieejami datu bāzē. Jaunajai kolonnai, kas satur marķieri nedrīkstēja saturēt NULL vērtības, tāpēc pie tabulas shēmas bija nepieciešams

definēt noklusējuma vērtību. Izmaiņas bija jāveic ne tikai tabulas shēmā, bet arī jāveido skripts, kas tika izpildīts pirms tabulu izveides, lai tiktu pievienota pareizā kolonna un pārveidots tabulas unikalitātes ierobežojums (unique constraint) (1).

Šis ierobežojums paredzēja to, ka vairāku kolonnu kopumam bija jābūt unikālam un jaunajam marķierim bija jābūt iekļautam. Skripta darbības rezultātā tika dzēsts šis ierobežojums, pievienota kolonna, kolonna papildināta ar pareizajām vērtībām un ierobežojums tika pārtaisīts iekļaujot jauno kolonnu.

Papildus kolonnas pievienošanai bija jāveic arī izmaiņas datu bāzes procedūrās. Vajadzēja pielāgot esošo finanšu plūsmas saskaņošanas procedūru tā, lai pie ievietošanas tiktu izvēlēti pareizie marķieri, kā arī izveidot jaunu procedūru, kas veiktu pilnīgi visu finanšu plūsmu saskaņošanu datu integritātes pārbaudei no aktuāru puses. Izstrādes laikā tika pievērsta pastiprināta uzmanība ātrdarbībai, jo vienā palaišanas reizē bija jāapstrādā vairāki miljoni rindu.

Šis uzdevuma daļa bija vissarežģītākā, jo sistēma ir ļoti kompleksa, tai ir ļoti daudz daļu un īpaši sarežģīts ir datu bāzes modelis, kas apraksta biznesa procesu, kā arī uzmanība ātrdarbībai un efektivitātei. Rezultātā tika izveidota procedūra, kas vairākus miljonus rindu apstrādāja 6 minūtēs izstrādes vidē un produkcijas vidē (datu skaits ir daudz lielāks) 10 minūtēs. Neskatoties uz grūtībām ar procedūru pielāgošanu un izveidi, pateicoties komandas atbalstam, iepriekšminētie šķēršļi neaizkavēja visu uzlabojumu piegādi produkcijā.

2.1.2. Cauruļvada izstrāde Azure Data Factory platformā

Projektā visa datu apstrāde tiek veikta ar cauruļvadiem (pipeline), kas ir izstrādāti Azure Data Factory platformā (2). Tā ir datu integrācijas platforma, kas ļauj veikt ETL\ELT procesus bez koda vai ar minimālu koda rakstīšanu. Pirms prakses es nebiju strādājis ar šo rīku, tāpēc bija nepieciešams papildināt, izmantojot oficiālo Microsoft dokumentāciju.

Cauruļvads Azure Data Factory kontekstā ir darbību kopums, kas definē datu apstrādes procesu noteiktā secībā. Pipeline sastāv no vairākām aktivitātēm (activities), kuras var ietvert datu kopēšanu, transformāciju, skriptu izpildi vai citu procesu izsaukšanu. Pipeline ļauj automatizēt datu apstrādes un integrācijas procesus starp dažādiem datu avotiem un sistēmām.

Mans uzdevums bija pielāgot esošos cauruļvadus, kas tika izmantoti dienu aprēķiniem, ka arī izveidot atsevišķu cauruļvadu, kura uzdevums bija:

- Pārbaudīt, vai diena, kad tas tiek palaists, atbilst pareizajai perioda slēgšanas dienai.
- Dzēst iepriekšējā perioda slēgšanas datus kopējā pieteikuma rezervju naudas plūsmas datus, kas marķēti kā analītiskie.
- Veikt finanšu plūsmu saskaņošanu, izsaucot procedūru.

Neskatoties uz to, ka platforma skaitās kā code-free, tomēr bija nepieciešams izstrādāt kodu, jo visi kalendārie dati tiek glabāti datu bāzē un tiek numurēti atbilstoši. Skripta uzdevums bija pareizi aprēķināt dienas un pieļaut darbību izpildi tikai konkrēti numurētās dienās. Skripts tika rakstīts T-SQL valodā, jo tas tika izpildīts datu bāzē.

Neskatoties uz to, ka iepriekš ar šo platformu es nebiju strādājis un par ETL\ELT procesiem man bija tikai teorētisks priekšstats, platformas kontrole, drag&drop (3) funkcionalitāte padarīja to intuitīvu un saprotamu. Galvenais, kas jāizprot bija biznesa vajadzības un kā tās tiek realizētas tehniski jau esošajā sistēmā.

Šī bija uzdevuma visvienkāršākā daļa un izdevās to paveikt daudz ātrāk nekā tika atvēlēts laika. ETL\ELT procesu realizācija Azure Data Factory platformā ietaupa daudz laika un ļauj integrēt un izmantot citus Azure mākoņskaitļošanas pakalpojumus, piemēram, Azure Functions, citus cauruļvadus, kā arī Teams paziņojumus un REST programmējamo lietotņu saskarņu (API) izsaukumus.

2.1.3. Procesa iespējošana procesu orķestratorā

Orķestrators ir sistēmas centrālā aplikācija, kas nosaka to, kad un kādi procesi tiek palaisti. Visas konfigurācijas un informācija par to, kādus procesus ir iespējams palaist. Ir galvenā procesu grafika tabula, kas nosaka, kurās dienās tiek palaists kāds process. Dati par procesiem tiek glabāt atsevišķi. Procesu smalkāk tiek iedalīti soļos un soļu konfigurācijā tiek norādīts, kas tieši tiek palaists – datu bāzes procedūra, izsaukts REST API galapunkts vai palaists cauruļvads (dati par šīm darbībām tiek glabāti atsevišķās datu bāzēs). Visi soļi ir piesaistīti vienam procesam un tiek secīgi numurēti.

Lai izpildītu uzdevuma nosacījumus man bija nepieciešams pievienot atsevišķus ierakstus datu bāzē tā, lai nokonfigurētu jaunu procesu un pievienotu esošajam procesam jaunu soli, kas bija izstrādātais cauruļvads.

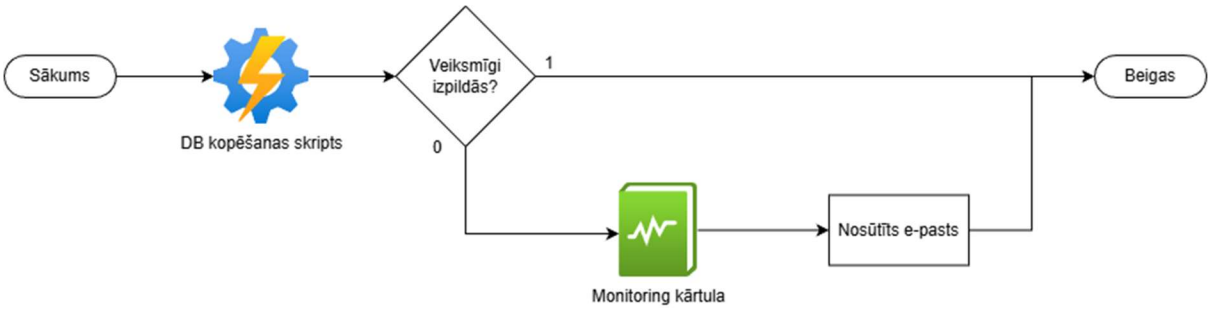
Viena no lietām, kas bija jāņem vērā bija līdzīga ar datu bāzes procedūras izstrādi – esošos datu integritātes saglabāšana. Tātad datiem būt jāparādās gan esošajā datu bāzē, kā arī gadījumā, ja datu bāze tiktu veidota no jauna, tad tiem datiem būtu jābūt pieejamiem arī tur. Kaut arī viss nepieciešamais kods, kas ievietoja, atjauninā ja vai dzēsa, bija izveidots bija svarīgi zināt un pievērst uzmanību tam kādas atslēgu vērtības jau tika izmantotas, jo tā kā dati ir statiski un tie tiek konfigurēti un papildināt palaižot kodu, tad atslēgas netika pieliktas automātiski – tās definēja izstrādātāji.

Šī daļa bija sarežģītāka, jo tā, kā es nebiju vienīgais komandā, kas veica procesu izstrādi, tad bieži pie palaišanas radās konflikti, kad atslēga tika jau izmantota. Pati izstrāde neaizņēma daudz laika, bet konfliktu risināšana gan datu bāzē un nereti arī versijkontrolē aizņēma vairāk laika nekā gribētos, bet uzdevums tika izpildīts.

2.2. Atkārtotā mēģinājuma automatizācija

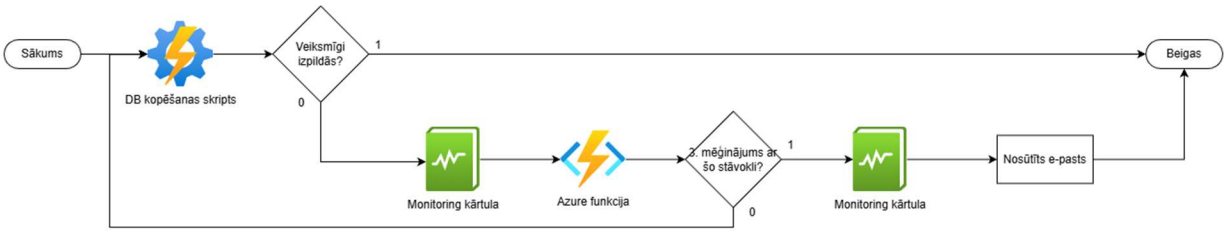
Katru nakti sistēmā tiek veikta datu bāzes produkcijas kopēšana, ko izmanto uzņēmuma analītiķi. Šo kopēšanu iespējo Azure Automation Account, kas ļauj izveidot dažāda veida kodu (runbooks), kas tiek palaists atbilstoši grafikam. Kodu šajā servisā ir iespējams rakstīt PowerShell, PowerShell Workflow un Python valodā.

Problēma, kas radās komandā bija tas, ka ne vienmēr šī kopēšana izdevās veiksmīgi. Vismaz 2-3 reizes mēnesī tā izgāzās un tad izstrādātājiem bija jāmēģina saprast, kas notika, kurā solī izgāzās un kāpēc. Par to, komanda uzzina, kad tiek atsūtīts automātisks e-pasts. Pēc izpētes tika veiktas atbilstošas darbības, lai šī kopija tiktu izveidota. Lai atvieglotu darbu un ļautu komandai pievērst uzmanību daudz svarīgākām lietām tika nolemts iespējot atkārtotos mēģinājumus.



2. attēls. Datu bāzes kopēšanas izpildes shēma.

Risinājuma izstrāde tika uzticēta man. Pēc iepriekšējas izpētes tika atklāts, ka iebūvētas atkārtotas palaišanas funkcionalitātes nav, tāpēc savā risinājumā es iekļāvu citus Azure mākoņpakalpojumus.



3. attēls. Datu bāzes kopēšanas uzlabotā risinājuma shēma.

Tā pamatā bija jāveic izmaiņas pašā kodā, kas tika palaists, kā arī jāizveido šis atkārtošanas mehānisms. Atkārtošanas mehānisma risinājumā tika ņemta vērā ne tikai paša atkārtošanās, bet arī resursu ekonomija, piemēram, ja kods izgāzās pie datu bāzes pārdēvēšanas, tad vajadzības veikt kopēšanu vēlreiz nav. Šī iemesla dēļ tika veidots risinājums, kas ļautu atsākt kopēšanu no tās daļas, kuras izgāzās, izņemot, ja kods izgāzās tieši pašas kopēšanas laikā, tad kopija tiek dzēsta un veidota no jauna.

2.2.1. Powershell runbook pielāgošana

Galvenā kopēšanas daļa ir Powershell skripts (4), kas pieslēdzas datu bāzu serverim un pa soļiem veic operatīvās datu bāzes kopēšanu. Skripta izpildē no sākuma tiek pārbaudīts, vai eksistē pagaidu datu bāze, ja tā ir, tad tiek dzēsta. Pēc pārbaudes tiek mērrogota (scale-up) datu bāze un palielināta jauda ātrdarbībai un veikto kopēšana pagaidu datu bāzē. Kad kopēšana ir pabeigta, tad tiek dzēsta iepriekšējā kopija, datu bāze tiek pārsaukta un pievienotas pielaides un atļaujas, lai biznesa lietotāji varētu veikt analīzi.

Tas, kas šajā skriptā bija jāpielāgo – pārbaudes pirms katra soļa, kas ļautu izpildīt soli tikai, ja iepriekšējais solis ir izpildīts. Tika pievienots arī parametrs, kas ļauj startēt skriptu no kāda konkrēta stāvokļa/soļa. Skriptā parametrs (5), kas nosaka sākuma stāvokli, tiek iestatīta noklusējuma vērtībai, lai skriptu automātiski varētu palaist Azure Automation Account.

Šīs izmaiņas bija diezgan vienkārši ieviest, kaut arī ar Powershell skriptošanu (6) man ir bijusi tikai neliela saskarsme, tomēr izmaiņas bija skaidras un saprotamas. Izmaiņu notestēšana izstrādes vidē pirms sistēmtestēšanas bija lieta, kas aizņēma visilgāk, jo datu bāzes kopēšana aizņēma aptuveni 1 stundu. Viena no grūtībām ar ko saskaros testēšanas laikā bija testēšana traucējumu scenārija gadījumā un, lai to veiktu tika izmantota Azure Chaos Studio, kas ļauj simulēt uz reālām instancēm traucējumus. Tas atviegloja situāciju gan man kā izstrādātājām, tā arī komandas kvalitātes nodrošināšanas inženierim, lai veiktu pārbaudes ne tikai veiksmīgajos scenārijos, bet arī tad, kad tiek pieredzēti infrastruktūras traucējumi.

2.2.2. Mākoņa infrastruktūras izstrāde

Lai iespējotu atkārtojumus, bija nepieciešams izstrādāt metrikas kārtulu (viens no monitoringa servisiem Azure), kas nostrādātu un izsauktu Azure Durable Function (7), kas ir Azure Functions paplašinājums (8), kas ļauj izveidot funkcijas (8., 9. pielikums) , kuras koordinē vairākus uzdevumus vai citas funkcijas vienotā darbplūsmā (5. un 6. pielikums). Atšķirībā no parastām serverless funkcijām, Durable Functions (9) saglabā izpildes stāvokli, tāpēc tās var droši darboties ilgāku laiku, gaidīt citus procesus vai atkārtot

darbības kļūmju gadījumā, kas ir noderīgi, lai saglabātu veikto stāvokļu un mēģinājumu stāvokli (state) (7. pielikums).

Tika ieviesti arī izņēmumi (exception), kas ļautu vienkāršāk veikt atkļūdošanu gadījumos, kad šis funkcijas mehānisms strādā nepareizi. Tika noklāts neesošas konfigurācijas gadījums (2. pielikums) un mākoņservisu nepieejamības gadījums, kā arī izņēmums (1. pielikums), kas nepieciešami loģikai – atkārtojumu biežuma robežas pārkāpšana: katram beidzamajam stāvoklim 3 un kopējais atkārtojumu skaits dienā 10, tas palaiž paziņojumu, ka mēģinājumi ir izsmelti un nepieciešama ir izstrādātāja iesaiste (3. pielikums).

Lai piekļūtu mākonim netika rakstīta atsevišķa loģika, bet tika izmantots Microsoft piedāvātais Azure SDK (4. pielikums), kurā ir ietvertas visas funkcijas, lai veiktu darbības ar Azure infrastruktūru. Funkcijā šīs Azure programmatūras izstrādes rīkkopas funkcionalitāte tika izmantota, lai iegūt pēdējo kopēšanas gadījumu Azure Account Automation un, lai nolasītu informāciju, kas Powershell skripta palaišanas laikā tiek izvadīta konsolē. No konsoles izvades tiek nolasīta informācija par to, kādā stāvoklī tika pabeigta kopēšana.

Funkcijas izstrāde bija diezgan vienkārša, jo tā tika rakstīta C# valodā (10). Funkcija veido orķestrācijas instances, kas katru reizi, kad izgāžas datu bāzes kopēšana, no sākuma tiek izdzēstas un tiek pievienots objekts ar pēdējo stāvokli, kas tika izvadīts pirms skripta izgāšanās. Tiek saglabāta arī tas, cik reizes tika izmēģināts atkārtot skripta izpildi ar šo stāvokli (11, 12). Vienīgais, kad tiek padots sākuma stāvoklis ir tad, ja skripts izgāžas datu bāzes pagaidu kopijas laikā, jo šajā gadījumā ir nepieciešams veidot pilnīgi jaunu kopiju un dzēst neizdevušos pagaidu.

2.2.3. Jaunas programmatūras palaišana mākoņa infrastruktūrā

Prakses vietā ir izveidota augsta līmeņa automātiska palaišanas sistēma, ka visus nepieciešamos sistēmas artefaktus un daļas kompilē un palaiž attiecīgajās vidēs automātiski, tāpēc bija svarīgi pārdomāt to, kas būtu jāmaina esošajā DevOps plūsmā, lai attiecīgās sistēmas daļas tiktu nokompilētas un palaistas droši.

Azure mākoņskaitļošanas infrastruktūras pakalpojumu lietošanu ir iespējams, iespējot ne tikai manuāli – izstrādātājs caur portālu izveido un nokonfigurē pakalpojuma izmantošanu (manā gadījumā paziņojuma kārtulu un funkcijas palaišanu (13)), bet to ir iespējams izdarīt arī definējot Bicep failā visas nepieciešamās konfigurācijas. Šī izstrādes daļa bija visvienkāršākā, jo es veidoju visus komponentus manuāli un, izmantojot Azure portālā sniegtās iespējas ģenerēju šos Bicep failus un ievietoju pareizajās direktorijās, kur DevOps palaišanas cauruļvads tās varēja nolasīt un palaist tos mākonī.

Funkciju izveidot bija grūtāk, jo tām ir konkrēts programmēšanas modelis, kas ir jāievēro, un saprast, kā sasniegt vēlamo rezultātu bija daudz grūtāk, tomēr tas tika paveikts. Šī bija vērtīga pieredze, jo tas deva iedvesmu idejai, kā aizvietot un uzlabot esošo orķestrācijas aplikāciju. Mākoņa servisu izstrādei ir nepieciešamas ne tikai programmēšanas prasmes, bet arī vispārīgas zināšanas par mākoņa infrastruktūru, tīkliem un skaitļošanas risinājumiem.

SECINĀJUMI UN PRIEKŠLIKUMI

Prakse uzņēmumā If P&C Insurance AS Latvijas filiāle man ir palīdzējusi attīstīties kā profesionālim un palīdzēja definēt jomu, kurā gribētu strādāt kā izstrādātājs, kā arī tika izpildīti visi uzdevumi un sasniegti visi mērķi. Pirmkārt, zināšanas datu bāzes koda optimizācijā ir nepieciešama daļa projektā, kas veic liela apjoma datu apstrādi; tās sevī ietver padziļinātas zināšana par to, ka darbojas programmatūra, kas izpilda SQL kodu. Otrkārt, darbs komandā un labas komunikācijas prasmes ir nepieciešamas efektīvam komandas darbam un efektīvai izstrādes piegādei. Treškārt, es secināju, ka ir ļoti vērtīgi ir būt ne tikai tehnoloģiski stipram, bet arī labi orientēties jomā, kurā strādā. Manā gadījumā tā bija finanšu un aktuārzinātnes joma.

Prakses laikā tika izpildīti un sasniegti visi uzdevumi, jo finanšu saskaņošanas funkcionalitāte un risinājums sistēmas noturības pret traucējumiem tika notestēts gan sistēmtestēšanas, gan integrācijas testēšanas, gan arī lietotāja akcepttestēšanas vidē, un aba izstrādātās funkcijas tika palaistas produkcijas vidē.

Veicot uzdevumus tika pamanītas lietas, kuras varētu uzlabot:

- **Tehniskā un izstrādes dokumentācija**. Kaut arī projektā ir lieliskas instrukcijas uzturēšanas aktivitātēm un skaidrojumi par biznesa loģiku, tomēr ir liels iztrūkums tieši izstrādes dokumentācijas jeb aprakstiem un instrukcijām – kādi standarti tiek ievēroti izstrādes laikā, kādi ir iestatījumi un konfigurācijas ieviešot funkcionalitāti ar mākoņpakalpojumu palīdzību. Tas ļautu ietaupīt laiku, kad tiek piegādāta jaunā funkcionalitāte uz dažādām vidēm.
- **Aktuālie vides artefakti**. Prakses laikā radās situācija, kad piegādes laikā uz produkciju, visa piegāde tika apstādināta vai izgāzās. Šajā gadījumā ilgāku laiku vide nav pieejama lietotājiem. Šo situāciju varētu atrisināt ar iepriekšējās vides versijas atgriešanu gadījumā, ja piegāde uz vidi izgāžas vai tiek atcelta.
- **Relīžu dokumentācija**. Komandas darbība attiecībā pret relīzēm ir ļoti atvieglota, jo komanda šajā ziņā nav atkarīga no citām komandā un tai ir

pieejami visi nepieciešamie rīki. Relīžu dokumentācija ļautu labāk izprast, kad kā un kāpēc tika veikti uzlabojumi, kas noder pie nākotnes funkcionalitātes risinājumu izstrādei.

Esmu apmierināts ar paveikto prakses laikā, un esmu pārliecinājies, ka spēju uzņemties iniciatīvu komandas darbā. Prakses laikā es biju spiests pamest komforta zonu un esmu daudz iemācījies.

IZMANTOTĀS LITERATŪRAS UN AVOTU SARAKSTS

1. https://learn.microsoft.com/en-us/azure/azure-sql/database/single-database-overview?view=azuresql; Microsoft; 27. janvāris 2026
2. https://learn.microsoft.com/en-us/azure/data-factory/introduction; Microsoft; 30. janvāris 2026
3. https://learn.microsoft.com/en-us/azure/data-factory/data-flow-create?tabs=data-factory Microsoft; 31. janvāris 2026
4. https://learn.microsoft.com/en-us/azure/data-factory/tutorial-hybrid-copy-data-tool; Microsoft; 4. februāris 2026
5. https://learn.microsoft.com/en-us/powershell/scripting/overview?view=powershell-7.5; Microsoft; 7. februāris 2026
6. http://learn.microsoft.com/en-us/powershell/scripting/learn/ps101/00-introduction?view=powershell-7.5; Microsoft; 9. februāris 2026
7. https://cdn.comparitech.com/wp-content/uploads/2018/08/Comparitech-Powershell-cheatsheet.pdf; Microsoft; 12. februāris 2026
8. https://learn.microsoft.com/en-us/azure/azure-functions/functions-overview; Microsoft; 3. marts 2026
9. https://learn.microsoft.com/en-us/azure/azure-functions/create-first-function-azure-developer-cli?tabs=linux%2Cget%2Cbash%2Cpowershell&pivots=programming-language-csharp; Microsoft; 10. marts 2026
10. https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-overview; Microsoft; 13. marts 2026
11. https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-isolated-create-first-csharp?pivots=code-editor-vscode; Microsoft; 16. marts 2026
12. https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-serialization-and-persistence?tabs=csharp-inproc; Microsoft; 20. marts 2026
13. https://learn.microsoft.com/en-us/azure/azure-functions/durable/durable-functions-configure-managed-identity; Microsoft; 21. marts 2026

PIELIKUMI

```
namespace If.SECRET.Azure.NightlyCopyHandler.Exceptions
{
 public class OutputCaptureFailedException : Exception
    {
        public OutputCaptureFailedException(string message) :
base(message) { }
    }
}
```

```
namespace If.SECRET.Azure.NightlyCopyHandler.Exceptions
{
    public class MissingConfigurationException : Exception
    {
        public MissingConfigurationException(string message) :
base(message) { }
    }
}
```

```
namespace If.SECRET.Azure.NightlyCopyHandler.Exceptions
{
    public class RetryLimitExceededException : Exception
    {
        public RetryLimitExceededException(string message) : base(message)
{ }
    }
}
```

```
using Azure;

namespace If.SECRET.Azure.NightlyCopyHandler.Extensions
{
    public static class AsyncPageableExtensions
    {
        public static async Task<List<T>> ToListAsync<T>(this
AsyncPageable<T> pageable, CancellationToken ct = default) where T :
notnull
        {
            var list = new List<T>();
            await foreach (var item in pageable.WithCancellation(ct))
            {
                list.Add(item);
            }

            return list;
        }
    }
}
```

```
using Azure.ResourceManager.Automation;

namespace If.SECRET.Azure.NightlyCopyHandler.Handlers
{
    public interface IAutomationHandler
    {
        Task<AutomationJobResource?> GetLatestJob();

        Task<List<string>> GetJobOutput(AutomationJobResource job);

        Task RestartJob(string environment, string step);
    }
}
```

```
using Azure;
using Azure.Identity;
using Azure.ResourceManager;
using Azure.ResourceManager.Automation;
using Azure.ResourceManager.Automation.Models;
using If.SECRET.Azure.NightlyCopyHandler.Exceptions;
using If.SECRET.Azure.NightlyCopyHandler.Extensions;

namespace If.SECRET.Azure.NightlyCopyHandler.Handlers
{
    public class AutomationHandler : IAutomationHandler
    {
        private const string CopySecretDb = "CopySecretDb";

        private readonly ArmClient _armClient;

        private readonly AutomationAccountResource _automationAccount;

        public AutomationHandler(string subscription, string
resourceGroup, string automationAccountName)
        {
            _armClient = new ArmClient(new DefaultAzureCredential());

            var accountId =
AutomationAccountResource.CreateResourceIdentifier(
                subscriptionId: subscription,
                resourceGroupName: resourceGroup,
                automationAccountName: automationAccountName
            );

            _automationAccount =
_armClient.GetAutomationAccountResource(accountId);
        }

        public async Task<AutomationJobResource?> GetLatestJob()
        {
            var jobs = await
_automationAccount.GetAutomationJobs().GetAllAsync().ToListAsync();
            var latestJob = jobs.Where(j => j.RunbookName ==
CopySecretDb).MaxBy(j => j.StartOn);

            if (latestJob is not null)
```

```
            {
                return await
_automationAccount.GetAutomationJobAsync(latestJob.Name);
            }

            return null;
        }

        public async Task<List<string>> GetJobOutput(AutomationJobResource
job)
        {
            var rawOutput = await job.GetOutputAsync();

            if (rawOutput is null)
                throw new OutputCaptureFailedException($"Output capture
for job {job.Data} failed.");

            var output =
rawOutput.Value.TrimEnd().Split("\r\n\r\n").ToList();
            return output;
        }

        public async Task RestartJob(string environment, string step)
        {
            var jobCollection = _automationAccount.GetAutomationJobs();
            var jobContent = new AutomationJobCreateOrUpdateContent
            {
                RunbookName = CopySecretDb
            };

            jobContent.Parameters.Add("Environment", environment);

            // If this is the last step before failure, then it means that
job failed during the copy phase.
            // Thus temp copy needs to be deleted before restart and job
started with default value.
            if (step != "CleanupDone")
            {
                jobContent.Parameters.Add("StartingState", step);
            }

            var createdJob = await jobCollection.CreateOrUpdateAsync(
                    WaitUntil.Started,
                    Guid.NewGuid().ToString(),
                    jobContent
```

```
                );
            }
        }
    }
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace If.SECRET.Azure.NightlyCopyHandler.Model
{
    public class Attempt
    {
        /// <summary>
        /// Last successfull step of the runbook.
        /// </summary>
        public string Step { get; set; }

        /// <summary>
        /// Date when attempts happened.
        /// </summary>
        public DateOnly CopyDate { get; private set; }

        /// <summary>
        /// Amount of retries attempted on the current step.
        /// </summary>
        public int Count { get; private set; }

        /// <summary>
        /// Total amount of retries tried in a day.
        /// </summary>
        public int TotalCount { get; private set; }

        public Attempt(string step, DateOnly copyDate, int count = 0, int
totalCount = 0)
        {
            Step = step;
            CopyDate = copyDate;
            Count = count;
            TotalCount = totalCount;
        }

        public void ResetCount()
        {
            Count = 0;
        }
```

```
        public void Add()
        {
            Count++;
            TotalCount++;
        }
    }
}
```

```
using Secrettle.Core.Logging;
using If.SECRET.Azure.NightlyCopyHandler.Exceptions;
using If.SECRET.Azure.NightlyCopyHandler.Handlers;
using If.SECRET.Azure.NightlyCopyHandler.Model;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Azure.Functions.Worker.Http;
using Microsoft.DurableTask.Client;
using Microsoft.DurableTask.Client.Entities;
using Microsoft.DurableTask.Entities;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Logging;
using System.Net;

namespace If.SECRET.Azure.NightlyCopyHandler;

public static class HandleCopyFailure
{
    private static readonly Attempt _startingState = new ("Started",
DateOnly.FromDateTime(DateTime.Now));

    /// <summary>
    /// Registers retry attempts and enforces limits.
    /// </summary>
    /// <returns>New attempt state.</returns>
    /// <exception cref="RetryLimitExceededException">Thrown when limit
per step or limit per day is exceeded and stops retry process.</exception>
    [Function(nameof(CountRetryAttempts))]
    public static Task CountRetryAttempts(
        [EntityTrigger] TaskEntityDispatcher dispatcher,
        FunctionContext ctx)
    {

        var logger = ctx.GetLogger(nameof(CountRetryAttempts));

        return dispatcher.DispatchAsync(operation =>
        {
            var state = operation.State.GetState<Attempt>();
            var input = operation.GetInput<Attempt>();

            if (state is null)
            {
                operation.State.SetState(_startingState);
                state = _startingState;
            }
```

```
            if (state.CopyDate != input?.CopyDate)
            {
                state = _startingState;
            }

            if (state.Step != input?.Step)
            {
                state.ResetCount();
                state.Step = input?.Step ?? _startingState.Step;
            }

            state.Add();

            if (state.Count > 3)
            {
                throw new RetryLimitExceededException(
                        $"Retry limit for the step {state.Step} has been
exceeded.");
            }

            if (state.TotalCount > 10)
            {
                throw new RetryLimitExceededException(
                    $"Retry limit for the date {state.CopyDate} has been
exceed.");
            }

            operation.State.SetState(state);

            return new(state);
        });
    }

    /// <summary>
    /// Http trigger for retry process start.
    /// </summary>
    [Function("NightlyCopyHandler_StartRetry")]
    public static async Task HttpStart(
        [TimerTrigger("%RetrySchedule%")] TimerInfo timerInfo,
        [DurableClient] DurableTaskClient client,
        FunctionContext executionContext)
    {
        Microsoft.Extensions.Logging.ILogger logger =
executionContext.GetLogger("NightlyCopyHandler_StartRetry");
```

```
        var handler =
executionContext.InstanceServices.GetRequiredService<IAutomationHandler>()
;
        var latestJob = await handler.GetLatestJob();

        if (latestJob is null)
        {
            return;
        }

        if (latestJob.Data.Status != "Failed")
        {
            return;
        }

        var output = await handler.GetJobOutput(latestJob);
        var finalState = output.LastOrDefault() ?? _startingState.Step;

        var executionDate =
DateOnly.FromDateTime(latestJob.Data.StartOn!.Value!.DateTime);

        var state = new Attempt(finalState, executionDate);
        var entity = new EntityInstanceId(nameof(CountRetryAttempts),
"currentState");
        await client.Entities.SignalEntityAsync(entity,
nameof(CountRetryAttempts), state);

        await handler.RestartJob("dev", finalState);
    }

    /// <summary>
    /// Http trigger to check current state of the retry process.
    /// </summary>
    [Function("NightlyCopyHandler_GetRetryState")]
    public static async Task<HttpResponseData> HttpGet(
        [HttpTrigger(AuthorizationLevel.Anonymous, "get")] HttpRequestData
req,
        [DurableClient] DurableTaskClient client)
    {
        var entityId = new EntityInstanceId(nameof(CountRetryAttempts),
"currentState");
        EntityMetadata<Attempt>? entity = await
client.Entities.GetEntityAsync<Attempt>(entityId);
```

```
        if (entity is null)
        {
            return req.CreateResponse(HttpStatusCode.NotFound);
        }

        var response = req.CreateResponse(HttpStatusCode.OK);
        await response.WriteAsJsonAsync(entity);

        return response;
    }

    /// <summary>
    /// Http trigger to reset state if not reset automatically.
    /// </summary>
    [Function("NightlyCopyHandler_ResetState")]
    public static async Task<HttpResponseData> HttpReset(
        [HttpTrigger(AuthorizationLevel.Anonymous, "post")]
HttpRequestData req,
        [DurableClient] DurableTaskClient client)
    {
        var entity = new EntityInstanceId(nameof(CountRetryAttempts),
"currentState");
        await client.Entities.SignalEntityAsync(entity,
nameof(CountRetryAttempts), _startingState);

        var response = req.CreateResponse(HttpStatusCode.OK);
        await response.WriteAsJsonAsync(entity);

        return response;
    }
}
```

```
var builder = FunctionsApplication.CreateBuilder(args);

builder.ConfigureFunctionsWebApplication();

builder.Services
    .AddApplicationInsightsTelemetryWorkerService()
    .ConfigureFunctionsApplicationInsights();

builder.Services.AddScoped<IAutomationHandler, AutomationHandler>(s =>
{
    var subscription = Environment.GetEnvironmentVariable("Subscription")
        ?? throw new MissingConfigurationException($"Value for
'Subscription' is missing.");

    var resourceGroup =
Environment.GetEnvironmentVariable("ResourceGroup")
        ?? throw new MissingConfigurationException($"Value for
'ResourceGroup' is missing.");

    var automationcAccount =
Environment.GetEnvironmentVariable("AutomationAccount")
        ?? throw new MissingConfigurationException($"Value for
'AutomationAccount' is missing.");

    return new AutomationHandler(subscription, resourceGroup,
automationcAccount);
});

builder.Build().Run();
```